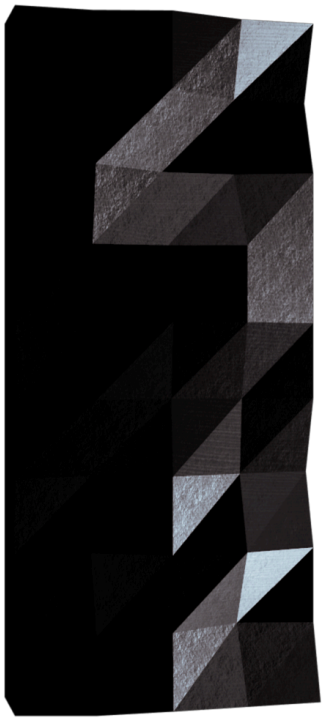


A-Maze-ing

This is the way

概要：自分だけの迷路生成器を作り、その結果を表示しよう！

バージョン：2.3

※ 本書は en.subject.pdf（Version 2.3）の日本語訳です。正式な要件は英語原文を優先してください。

目次

I はじめに 3

II AI に関する指示 4

III 共通の指示 6

III.1 一般ルール 6

III.2 Makefile 6

III.3 追加ガイドライン 7

IV 必須パート 8

IV.1 概要 8

IV.2 使い方 8

IV.3 設定ファイルの形式 8

IV.4 迷路の要件 9

IV.5 出力ファイルの形式 10

V 視覚的表示 12

VI コード再利用性の要件 14

VII README の要件 15

VIII ボーナス 17

IX 提出とピア評価 18

第 I 章

はじめに

迷路は何千年もの間、人々を魅了してきました。ギリシャ神話で、ダイダロスがミノタウロスを閉じ込めるために築いたという伝説のクノッソスの迷宮から、現代のパズル本やビデオゲームに至るまで、迷路は常に神秘・挑戦・巧みな設計の象徴でした。コンピュータサイエンスにおいて、迷路生成は単なる遊びではありません。アルゴリズム、乱数、グラフ理論の実践的な応用です。迷路生成に使われる有名なアルゴリズム——プリム法、クラスカル法、再帰的バックトラッキングなど——は、ネットワーク設計やプロシージャルなコンテンツ生成といった現実の問題にも使われています。興味深いことに、完全迷路（任意の 2 点間の経路がただ一つしかない迷路）は、グラフ理論における全域木と直接対応しています。迷路を作ること、とりわけ視覚化して他人と共有できる迷路を作ることは、コンピュータが混沌から構造を生み出す仕組みを探究しつつ、ちょっと楽しむのにうってつけの方法です。

> *「迷宮とは迷うための場所ではなく、道を見つけるための場所である。」*
> —— *作者不詳*

第 II 章

AI に関する指示

● 背景

学習の過程で、AI はさまざまな作業を手助けしてくれます。AI ツールが持つ多様な能力と、それが自分の作業をどう支えられるかを、時間をかけて探ってみてください。ただし、常に慎重に向き合い、結果を批判的に評価してください。コードであれ、ドキュメントであれ、アイデアであれ、技術的な説明であれ、自分の質問が適切に組み立てられていたか、生成された内容が正確かどうかを完全に確信することはできません。仲間（ピア）は、ミスや見落としを防ぐための貴重な存在です。

● 主なメッセージ

- ☛ 繰り返しの作業や退屈な作業を減らすために AI を使う。
- ☛ 将来のキャリアに役立つプロンプト作成スキルを、コーディング・非コーディングの両面で身につける。
- ☛ AI システムの仕組みを学び、よくあるリスク・バイアス・倫理的問題を予測し回避できるようにする。
- ☛ 仲間と協働しながら、技術スキルとパワースキル（対人・思考スキル）の両方を磨き続ける。
- ☛ 完全に理解し、責任を持てる AI 生成コンテンツだけを使う。

● 学習者のルール

- AI ツールを探究し、その仕組みを理解する時間を取ること。そうすることで倫理的に使い、潜在的なバイアスを減らせる。
- プロンプトを書く前に、自分の問題についてよく考えること。そうすれば、正確な語彙を使って、より明確で詳細かつ的確なプロンプトを書ける。
- AI が生成したものはすべて、体系的に確認・レビュー・疑問視・テストする習慣をつけること。
- 常にピアレビューを求めること。自分だけの検証に頼らない。

● このフェーズで得られるもの

- 汎用的なプロンプトスキルと、分野特化のプロンプトスキルの両方を身につける。
- AI ツールを効果的に使い、生産性を高める。
- 計算論的思考、問題解決力、適応力、協働力を引き続き強化する。

● コメントと例

- 試験や評価など、本当に理解していることを示さなければならない場面には定期的に遭遇します。備えておきましょう。技術スキルと対人スキルの両方を磨き続けてください。
- 自分の考えを説明したり仲間と議論したりすると、理解の穴が見えてくることがよくあります。ピアラーニングを優先しましょう。
- AI ツールはあなた固有の文脈を知らないことが多く、一般的な回答をしがちです。同じ環境を共有する仲間のほうが、より的確で正確な洞察をくれます。
- AI は最も「ありそうな」答えを生成する傾向がありますが、仲間は別の視点や貴重なニュアンスを提供してくれます。品質のチェックポイントとして仲間を頼りましょう。

> **✓ 良い例**
>
> AI に「ソート関数はどうテストすればいい？」と聞くと、いくつかのアイデアが返ってくる。それを試し、結果を仲間と一緒に見直す。二人でやり方を改善していく。

> **✗ 悪い例**
>
> AI に関数をまるごと書かせ、プロジェクトにコピペする。ピア評価のとき、それが何をしているのか、なぜそうなのかを説明できない。信用を失い、課題も不合格になる。

> **✓ 良い例**
>
> AI を使ってパーサーの設計を手伝ってもらう。その後、仲間と一緒にロジックを一つずつ確認する。バグを 2 つ見つけ、二人で書き直す——より良く、よりきれいに、そして完全に理解した状態で。

> **✗ 悪い例**
>
> プロジェクトの重要な部分のコードを Copilot に生成させる。コンパイルは通るが、パイプをどう処理しているのか説明できない。評価のときに根拠を示せず、課題は不合格になる。

第 III 章

共通の指示

III.1　一般ルール

- プロジェクトは **Python 3.10 以降**で書くこと。
- プロジェクトは **flake8** のコーディング規約に従うこと。
- 関数はクラッシュを避けるため、例外を適切に処理すること。起こりうるエラーは `try-except` ブロックで扱うこと。ファイルや接続などのリソースには、自動的に後始末されるようコンテキストマネージャを優先して使うこと。レビュー中に未処理の例外でプログラムがクラッシュした場合、そのプログラムは動作しないものとみなされる。
- すべてのリソース（ファイルハンドル、ネットワーク接続など）はリークを防ぐよう適切に管理すること。可能な限りコンテキストマネージャを使って自動的に扱うこと。
- 関数の引数、戻り値、および必要に応じて変数に型ヒントを付けること（`typing` モジュールを使用）。静的型チェックには **mypy** を使うこと。すべての関数が mypy をエラーなしで通過しなければならない。
- 関数とクラスには PEP 257 に従った docstring（Google スタイルや NumPy スタイルなど）を付け、目的・引数・戻り値を記述すること。

III.2　Makefile

よく使う作業を自動化するため、プロジェクトに Makefile を含めること。Makefile には以下のルールが必要である（必須の `lint` には指定のフラグが含まれる。より厳密なチェックのため `--strict` も試すことを強く推奨する）。

- **install**：pip、uv、pipx、その他任意のパッケージマネージャを使ってプロジェクトの依存関係をインストールする。
- **run**：プロジェクトのメインスクリプトを実行する（例：Python インタプリタ経由）。
- **debug**：Python 組み込みのデバッガ（例：pdb）を使い、メインスクリプトをデバッグモードで実行する。
- **clean**：一時ファイルやキャッシュ（例：`__pycache__`、`.mypy_cache`）を削除し、プロジェクト環境をきれいに保つ。
- **lint**：`flake8 .` と `mypy . --warn-return-any --warn-unused-ignores --ignore-missing-imports --disallow-untyped-defs --check-untyped-defs` を実行する。
- **lint-strict**（任意）：`flake8 .` と `mypy . --strict` を実行する。

III.3　追加ガイドライン

- プロジェクトの機能を検証するテストプログラムを作ること（提出・採点の対象外）。pytest や unittest などのフレームワークを使い、エッジケースも網羅した単体テストを書くこと。
- Python の生成物を除外する `.gitignore` ファイルを含めること。
- 開発中は依存関係を分離するため、仮想環境（例：venv や conda）の使用を推奨する。

> プロジェクト固有の追加要件がある場合は、このセクションの直後に記載される。

第 IV 章

必須パート

IV.1　概要

設定ファイルを受け取って迷路を生成し、その迷路を 16 進数による壁表現でファイルに書き出す迷路生成器を Python で実装します。迷路は完全迷路（入口と出口の間の経路がただ一つ）である場合もあります。あわせて迷路の視覚的表示を用意し、生成ロジックを後で再利用できるようにコードを構成してください。

IV.2　使い方

プログラムは次のコマンドで実行できなければならない：

```
python3 a_maze_ing.py config.txt
```

- `a_maze_ing.py` はメインプログラムのファイルである。**この名前を使わなければならない。**
- `config.txt` は唯一の引数である。迷路生成のオプションを定義するプレーンテキストファイルで、ファイル名は別のものでもよい。

プログラムは、不正な設定、ファイルが見つからない、構文の誤り、実現不可能な迷路パラメータなど、あらゆるエラーを適切に処理しなければならない。予期せずクラッシュしてはならず、常にユーザーに分かりやすいエラーメッセージを表示しなければならない。

IV.3　設定ファイルの形式

設定ファイルは 1 行につき 1 つの `KEY=VALUE` の組を含むこと。

`#` で始まる行はコメントであり、無視しなければならない。
以下のキーは必須である：

キー	説明	例
`WIDTH`	迷路の幅（セル数）	`WIDTH=20`
`HEIGHT`	迷路の高さ	`HEIGHT=15`
`ENTRY`	入口の座標 (x,y)	`ENTRY=0,0`
`EXIT`	出口の座標 (x,y)	`EXIT=19,14`

`OUTPUT_FILE`	出力ファイル名	`OUTPUT_FILE=maze.txt`
`PERFECT`	完全迷路にするか	`PERFECT=True`

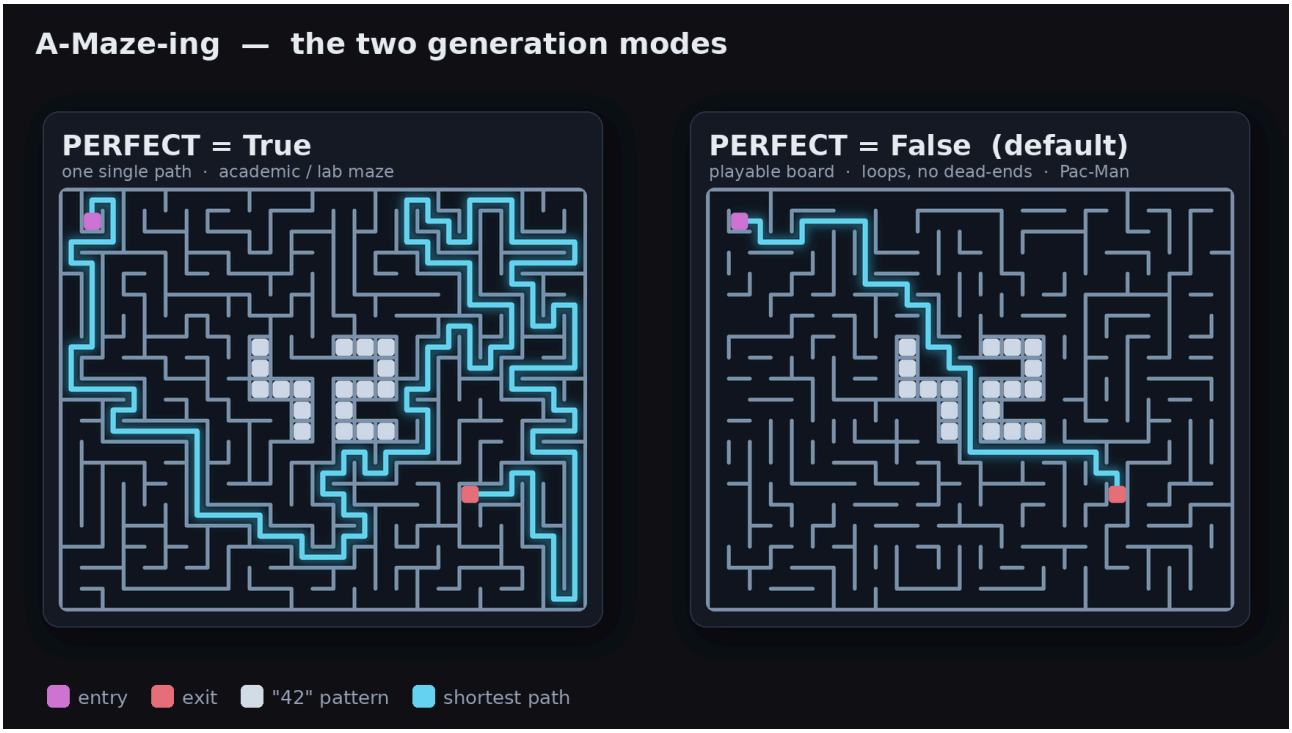
有用であれば、追加のキー（例：シード、アルゴリズム、表示モード）を加えてもよい。

デフォルトの設定ファイルを Git リポジトリに置いておかなければならない。

IV.4　迷路の要件

- 迷路はランダムに生成されなければならないが、**シードによる再現性が必須**である。
- 迷路の各セルは、東西南北の各方位に 0〜4 枚の壁を持つ。
- 迷路は有効でなければならない。すなわち：
  - 入口と出口が存在し、互いに異なり、迷路の範囲内にあること。
  - 構造が完全な連結性を保証し、孤立したセルがないこと（「42」パターンを除く。後述）。
  - 入口と出口は特定のセルなので、外周には壁がなければならない。
  - 生成データは整合していなければならない：隣り合うセルどうしは、壁があるなら同じ壁を共有していること。たとえば、あるセルの東側に壁があるのに、その壁の向こうのセルの西側に壁がない、という状態は禁止である。
- 迷路に広い開けた空間があってはならない。**通路の幅は 2 セルを超えてはならない。**
  たとえば 2×3 や 3×2 の開けた空間は許されるが、3×3 の開けた空間は決して許されない。
- 視覚的に表示したとき（後述）、迷路には、完全に閉じた複数のセルで描かれた「42」が見えていなければならない。
- `PERFECT` フラグが有効な場合、迷路は入口と出口の間にちょうど 1 本の経路を持たなければならない（すなわち完全迷路：ループが一切ない）。
- `PERFECT` フラグが無効な場合（デフォルト）、迷路は代わりに、パックマン風ゲームでそのまま使える盤面でなければならない。具体的には：
  - すべての通路に到達可能であること（完全な連結性）。これにより盤面全体をパックガム（エサ）で埋めても、クリア可能な状態が保たれる。
  - 四隅と中央が開いた通路であること（ゴーストとパワーエサは四隅に、プレイヤーは中央からスタートする）。
  - 少なくとも 2 本の独立したルート（ループ）があり、追われているプレイヤーに常に逃げ道があること（したがって、完全迷路や、完全迷路から壁を 1 枚取り除いただけのもの（ループが 1 つだけ）は、このモードでは認められない）。
  - 行き止まりはまれであるべき（2〜3 個程度は許容される）。行き止まりがまったくない盤面が理想であり、ボーナスとして評価される（「ボーナス」の章を参照）。

> 迷路のサイズが小さすぎるなどで「42」パターンを配置できない場合は、省略してもよい。その場合はコンソールにエラーメッセージを表示すること。



同じグリッド上での 2 つの生成モード：`PERFECT=True` では曲がりくねった経路が 1 本だけになる（それ以外の通路はすべて行き止まり）。一方、デフォルトの `PERFECT=False` では少なくとも 2 本の独立したルートが開いており、追われているプレイヤーに常に逃げ道がある。どちらも最短経路を強調表示している。
（図中の凡例：entry＝入口、exit＝出口、"42" pattern＝「42」パターン、shortest path＝最短経路。左：単一経路・学術／実験用迷路、右：プレイ可能な盤面・ループあり・行き止まりなし・パックマン）

IV.5　出力ファイルの形式

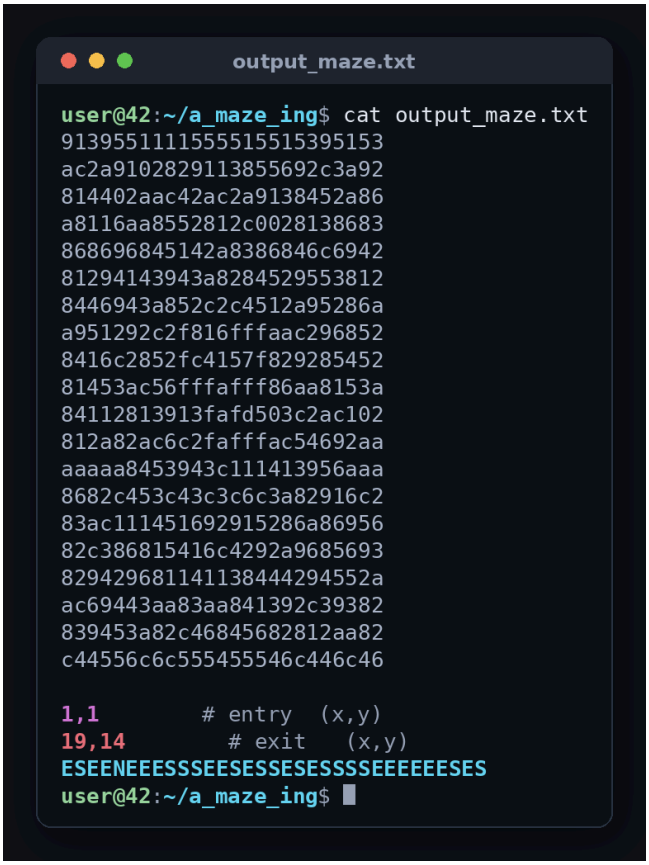
迷路は、**1 セルにつき 1 桁の 16 進数**を使って出力ファイルに書き出さなければならない。各桁は、どの壁が閉じているかを表す：

ビット	方向
0（LSB）	北（North）
1	東（East）
2	南（South）
3	西（West）

- 壁が閉じていればそのビットは 1、開いていれば 0 になる。
  例：`3`（2 進数 `0011`）は南と西の壁が開いていることを意味する。また `A`（2 進数 `1010`）は東と西の壁が閉じていることを意味する。

- セルは迷路の行ごとに格納し、ファイルの 1 行に迷路の 1 行分を書く。
- 空行を 1 行挟んだあと、以下の 3 つの要素を 3 行に分けて出力ファイルに書き込む：
    - 入口の座標、出口の座標、そして入口から出口までの有効な最短経路（`N`、`E`、`S`、`W` の 4 文字で表す）。
- すべての行は `\n` で終わること。

専用の設定ファイルと組み合わせることで、この出力ファイルは Moulinette（自動採点システム）によって自動テストされる可能性がある。この課題には解析スクリプト（`maze_analyzer.py`）も付属している。このスクリプトは出力ファイルを受け取り、壁のエンコードが整合しているか（共有する壁が隣り合う両セルで同じようにエンコードされているか）を確認し、迷路が完全迷路か（`PERFECT=True` で要求される単一経路）、それともプレイ可能な盤面か（デフォルトで要求される、「42」パターン以外に行き止まりがなく、複数の独立したルートを持つ盤面）を報告する。生成器が設定ファイルで要求されたモードを出力しているかの確認に使ってよい。



出力ファイルの例
（`# entry (x,y)` ＝入口の座標、`# exit (x,y)` ＝出口の座標）

第 V 章

視覚的表示

プログラムは、次のいずれかの方法で迷路を視覚的に表示する手段を提供しなければならない：

- ターミナルでの ASCII 描画、または
- MiniLibX（MLX）ライブラリを使ったグラフィカル表示。

表示では、壁、入口、出口、解答経路がはっきり分かるようにすること。

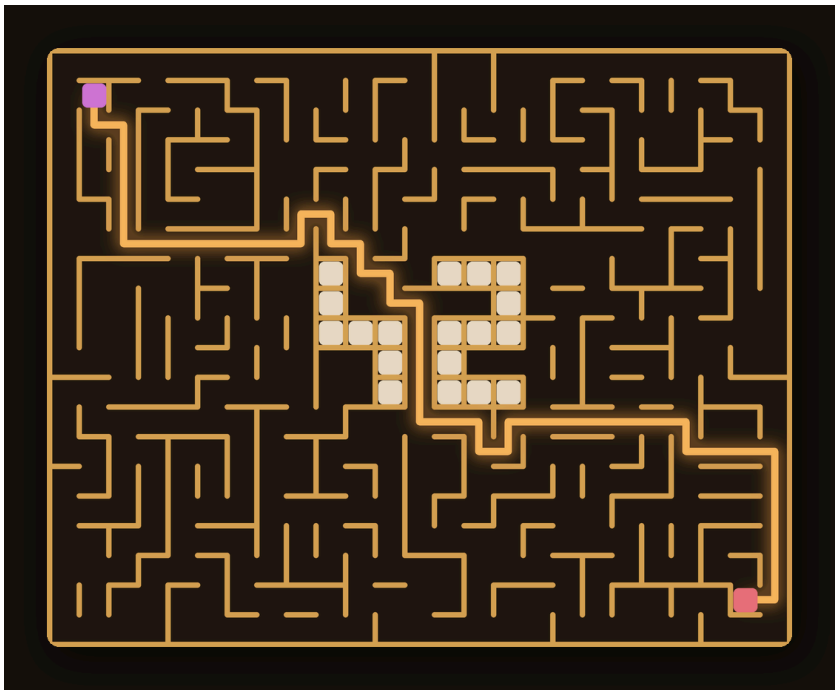
プログラムの実行中にユーザー操作ができなければならず、少なくとも以下の操作を含むこと：

- 新しい迷路を再生成して表示する。
- 入口から出口までの有効な最短経路の表示／非表示を切り替える。
- 迷路の壁の色を変更する。
- 任意：「42」パターンの表示に特定の色を設定する。

ユーザー操作を追加してもよい。



迷路のデフォルトのターミナル描画



別の迷路、最短経路の表示、壁の色の変更



MLX を使った迷路の描画

第 VI 章

コード再利用性の要件

迷路生成は、将来のプロジェクトでインポートできる独立したモジュールの中に、**単一のクラス**（例：`MazeGenerator`）として実装しなければならない。

以下の方法を説明する短いドキュメントを用意すること：

- 生成器のインスタンスを作って使う方法（少なくとも基本的な例を 1 つ）。
- カスタムパラメータ（例：サイズ、シード）を渡す方法。
- 生成された構造にアクセスする方法、および少なくとも 1 つの解答（経路）にアクセスする方法。

> 迷路生成モジュールは迷路の構造へのアクセスを提供するが、その形式は出力ファイルと同じである必要はない。

この再利用可能なモジュール全体（コードとドキュメント）は、後で pip でインストールできる単一のファイルとして用意しなければならない。
このパッケージ名は `mazegen-*` とし、そのファイルは Git リポジトリのルートに置くこと。
Python パッケージの標準ビルドで生成される `.tar.gz` と `.whl` のどちらの拡張子も認められる。
完全なファイル名の例：`mazegen-1.0.0-py3-none-any.whl`

パッケージのビルドに必要なものはすべて Git リポジトリに含めること。評価時には次のことが求められる：virtualenv などの環境で必要なツールをインストールし、ソースからパッケージを再ビルドする。

この迷路生成器は後のプロジェクトで再利用されることを前提としているため、リポジトリのルートに `LICENSE.md` ファイルを置き、コードを再利用する際のライセンスを明記しなければならない。このライセンスを選んで書くことも課題の一部である。これはソフトウェアライセンスと知的財産に初めて触れる機会となる。選ぶライセンスは、この迷路生成器を土台にする後のプロジェクトによる再利用と配布を明示的に許可するものでなければならない。

メインの `README.md` ファイル（再利用モジュールの一部ではない）にも、この短いドキュメントを含めること。

第 VII 章

README の要件

Git リポジトリのルートに `README.md` ファイルを置かなければならない。その目的は、プロジェクトをよく知らない人（仲間、スタッフ、採用担当者など）が、プロジェクトの内容、実行方法、関連情報の入手先を素早く理解できるようにすることである。

`README.md` には少なくとも以下を含めること：

- 最初の 1 行目は**斜体**で、次の文言とすること：
  This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].
  （訳：このプロジェクトは 42 のカリキュラムの一環として <login1>… によって作成された。※ 文言は英語のまま記載すること）
- 「Description（説明）」セクション：プロジェクトの目的と簡単な概要を含め、プロジェクトを明確に紹介する。
- 「Instructions（手順）」セクション：コンパイル、インストール、実行に関する必要な情報を記載する。
- 「Resources（参考資料）」セクション：テーマに関連する定番の参考資料（ドキュメント、記事、チュートリアルなど）を挙げるとともに、AI をどのように使ったか——どの作業に、プロジェクトのどの部分で使ったか——を記載する。

➠ プロジェクトによっては追加のセクションが求められる場合がある（例：使用例、機能一覧、技術的な選択など）。

必要な追加項目は以下に明記する。

- 設定ファイルの完全な構造と形式。
- 選んだ迷路生成アルゴリズム。
- そのアルゴリズムを選んだ理由。
- コードのどの部分が再利用可能で、どう再利用するか。
- チームとプロジェクト管理について：
  - 各メンバーの役割。
  - 当初の計画と、それが最後までどう変化したか。
  - うまくいったことと、改善できること。
  - 特定のツールを使ったか？ 使ったならどれか？

高度な機能（複数のアルゴリズム、表示オプションなど）を実装した場合は、この `README.md` に記載すること。

> 英語での記述を推奨する。代わりに、所属キャンパスの主要言語を使ってもよい。

第 VIII 章

ボーナス

プロジェクトにさまざまなボーナスを加えてよい。以下はその例である：

- 行き止まりがまったくないデフォルト（非完全）迷路：完全に「ブレイド化（braided）」された盤面で、追われているプレイヤーがどこでも追い詰められない（付属の解析スクリプトで `--max-dead-ends 0` を指定して確認できる）。
- 複数の迷路生成アルゴリズムに対応する。
- 迷路生成中のアニメーションを追加する。

第 IX 章

提出とピア評価

いつもどおり、課題を Git リポジトリに提出すること。ディフェンス（評価面談）で評価されるのは、リポジトリ内の成果物のみである。ファイル名が正しいか、ためらわずに再確認すること。

評価中に、プロジェクトの**簡単な修正**を求められることがある。ちょっとした挙動の変更、数行のコードの記述や書き換え、あるいは簡単に追加できる機能などである。

この手順はすべてのプロジェクトに当てはまるわけではないが、評価ガイドラインに記載されている場合は**備えておかなければならない**。

この手順は、プロジェクトの特定部分を本当に理解しているかを確認するためのものである。修正は任意の開発環境（普段の環境など）で行ってよく、評価の一部として特定の制限時間が定められていない限り、数分で終えられる程度のものである。
たとえば、関数やスクリプトの小さな変更、表示の修正、新しい情報を格納するためのデータ構造の調整などを求められることがある。

詳細（範囲、対象など）は評価ガイドラインで指定され、同じプロジェクトでも評価ごとに異なる場合がある。